Break It, Then Build It

A SmartLock Cryptographic System Engagement

Muhammad Ali

Roll No: **23i-0008** · Section: **E**

CS3002 — Information Security · Assignment 2

Recovered flag: `FLAG{e29a590b9abc0b29}`

Exact build commands

```
# v1 tools (what I attacked) and my instance
make -C smartlock
cd smartlock && ./instance_generator 23i-0008      # writes instance.txt

# Part A exploits (each builds standalone)
g++ -std=c++17 -O2 -Wall exploits/23i-0008-E-TASKA3-V1.cpp -o taska3 -lcrypto
#   (TASKA1,2,4,5 build the same way; shared header 23i-0008-E-TASKA-
common.hpp)

# Part C v2 (my securelink2.cpp linked into the provided scaffold)
make -C v2_skeleton                                  # builds lock_server2,
hub_client2

# Part D record benchmark
g++ -std=c++17 -O2 -Wall performance/23i-0008-E-TASKD-V1.cpp securelink2.cpp
-Iperformance -o taskd -lcrypto
```

Toolchain: g++ 15.2 (C++17), OpenSSL 3.6, Kali Linux (6.19 x86_64), Intel i7-9750H.

Scope and ethics

All capture, interception, spoofing and exploitation described here was carried out entirely inside an isolated lab against my own instance of SmartLock, which I generated from my own roll number (`23i-0008`). The lock and hub were run as local processes and the on-path attacker was modelled with a local TCP relay (the two-VM "local relay" option in the Lab Setup Guide); all traffic was captured on the loopback interface `lo`. Nothing was ever bridged to the campus network or the internet, and no system other than my own SmartLock instance was touched. The MITM, downgrade and replay techniques here are powerful and are practised only so that the corrected design in Parts B–C can be justified and tested. Every recovered value in this report belongs to my own instance and was obtained from the wire, not read from `instance.txt`.

Part A — Breaking SmartLock v1

A.1 Recovered secrets

All four values were recovered from a single captured session of my own instance (`evidence/23i-0008-E-v1-normal.pcap` → reassembled to `evidence/v1_stream.txt`). Each exploit reads the capture; none hard-codes an instance value.

Key	Recovered value	How (exploit)
`flag`	`FLAG{e29a590b9abc0b29}`	TASKA3 (keying) / TASKA1 (downgrade) — decrypted greeting
`recovered_dh_b`	`405261734119`	TASKA3 — baby-step giant-step discrete log; verified $g^b \bmod p = B$
`cracked_pin`	`violet102`	John the Ripper on captured MD5 pinhash (wordlist + `?d?d?d`)
`nonce_xor_hex`	`0335282620446c382a3f2d6473180f1e1b2f0410561e`	TASKA4 — greeting ⊕ STATUS-reply (two-time pad)

DH group of my instance (read from the DH message): p = 1069135286239, g = 2, B = 705456199848, A = 277461206451. Check: $2^{405261734119} \bmod 1069135286239 = 705456199848 = B$. ✓

A.2 Five-stage exploit demonstrations

Stage 1 — Negotiation: downgrade (TASKA1)

Weakness. The suite list is sent in clear, nothing binds it, and `NULL` (no encryption) is an offered suite. An on-path attacker rewrites the hub's `HELLO` to offer only `NULL`; the lock then has no strong option and selects it, so every record — including the greeting carrying the flag — is plaintext. **Impact / property:** **confidentiality** (and channel integrity — the tampering is undetected).

Evidence (attacker relay rewrites the line in flight; the flag appears in clear):

```
[relay] rewrote HELLO -> suites:NULL    type:HELLO
[relay] record plaintext: Paired with SmartLock. Pairing secret: FLAG{e29a590b9abc0b29}
[relay] *** FLAG = FLAG{e29a590b9abc0b29} ***
[hub]  negotiated suite: NULL
```

Run: `./taska1 8888 127.0.0.1 9094` then `./hub_client instance.txt 127.0.0.1 8888`. Capture: `evidence/23i-0008-E-A1-downgrade.pcap`.

Stage 2 — Authentication: impersonation / MITM (TASKA2)

Weakness. The lock's certificate is self-signed and the hub never validates it, and the DH values are unsigned. The attacker runs *two* key exchanges — one with the hub, one with the lock — holding a different key with each side, and relays (and can rewrite) everything. It does *not* downgrade: the session stays AES-256-GCM and the attacker still reads it all, proving encryption without authentication is worthless. **Impact / property:** **authentication** (then confidentiality + integrity).

```
[mitm] suite kept strong: AES256-GCM
[mitm] two distinct keys established:
       k(hub)  = ...c6a956cb0d
       k(lock) = ...0649b52599
[mitm] GREETING (read in the middle): Paired with SmartLock. Pairing secret:
FLAG{e29a590b9abc0b29}
# with --tamper : the hub asked STATUS, the attacker forged UNLOCK ->
[mitm] TAMPER: rewriting 'STATUS' -> 'UNLOCK'     [lock] DOOR OPENED (unlock #1)
```

Run: `./taska2 8889 127.0.0.1 9094 [--tamper]`. Capture: `evidence/23i-0008-E-A2-mitm.pcap`.

Stage 3 — Keying: discrete log, no KDF, static exponent (TASKA3)

Weakness. The DH prime is only ~40 bits, so the discrete log is solvable in under a second by baby-step giant-step ($b = i \cdot m + j$, $m \approx \sqrt{p}$). The lock also reuses a static exponent b (no forward secrecy) and uses the *raw* shared secret as the AES key (no KDF). Recovering b once rebuilds the exact key for every session and decrypts the greeting. **Impact / property:** **confidentiality of the key & forward secrecy**.

```
$ ./taska3 v1_stream.txt
[+] recovered_dh_b = 405261734119
[*] check g^b mod p = 705456199848  (must equal B = 705456199848)  -> OK
[*] record key     = ...0949afab42
[+] greeting plaintext: Paired with SmartLock. Pairing secret: FLAG{e29a590b9abc0b29}
```

Stage 4 — Message protection: nonce reuse (TASKA4)

Weakness. Every record is AES-GCM under the *same* (key, nonce): the static nonce is reused and sent in clear. Two records then share one keystream, so C1 ⊕ C2 = P1 ⊕ P2 (two-time pad). The greeting's known prefix recovers the other plaintext with no key at all. **Impact / property:** **confidentiality (and integrity — GCM forbidden attack)**.

```
$ ./taska4 v1_stream.txt
[+] SAME nonce reused -> two-time pad applies
[+] P1 xor P2 (nonce_xor_hex) = 0335282620446c382a3f2d6473180f1e1b2f0410561e
[+] recovered STATUS-reply plaintext: "STATE LOCKED unlocks=0"
```

Stage 5 — Teardown: replay (TASKA5)

Weakness. Records carry no sequence number, the lock keeps no anti-replay state, and the close is unauthenticated. Because b and the nonce are static, a replayed handshake reproduces the same key, so a captured `UNLOCK` record re-opens the door on a fresh connection — a permanent skeleton key. **Impact / property:** **integrity / freshness (and availability)**.

```
=== replay #1 ===   lock's B this session = 705456199848 (captured B) -> SAME (static
exponent!)
[+] REPLAY ACCEPTED. lock reply: DOOR OPENED (unlock #2)
=== replay #2 ===   [+] REPLAY ACCEPTED. lock reply: DOOR OPENED (unlock #3)
=== normal session after replays ===  [lock] STATE UNLOCKED unlocks=3   # counter climbed
```

Run: `./taska5 v1_stream.txt 127.0.0.1 9094` (re-run to watch the counter climb). Capture: `evidence/23i-0008-E-A5-replay.pcap`.

Part B — Designing SmartLock v2

Threat model

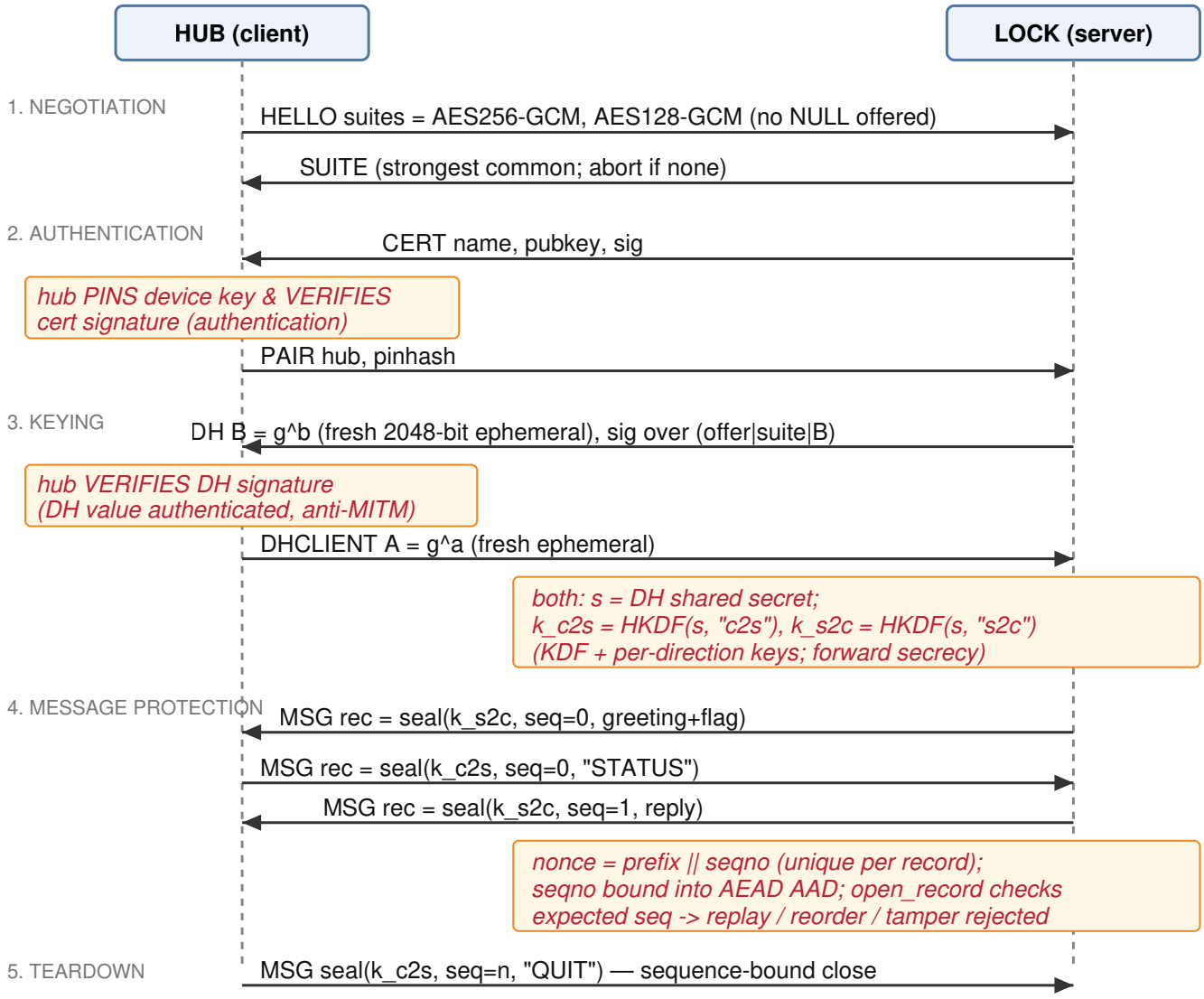
Attacker capabilities. A Dolev–Yao network attacker on the local link: it can *see* every byte (the link is the capture surface), *drop, reorder, replay and modify* any message, and *inject* its own, and it can take an on-path (MITM) position between hub and lock. It may capture any number of past sessions and work on them offline. **What it lacks.** It does not know the device's RSA private key, the hub's pre-shared/pinned copy of the device public key, or the per-session ephemeral DH secrets. It cannot break AES-256-GCM, SHA-256, HKDF or 2048-bit DH by computation. **Security goals v2 must meet.** (G1) confidentiality of commands and of the flag; (G2) integrity of every record; (G3) authentication of the lock to the hub; (G4) freshness — no replay or reorder; (G5) forward secrecy — a later key compromise must not expose past sessions; (G6) fail-closed negotiation — never silently fall back to weak/no crypto.

Per-stage mechanisms (each tied to the Part A attack it defeats)

Stage	Mechanism in v2	Defeats
1 Negotiation	**Strong-only suite set + fail-closed policy.** `strong_suites()` excludes NULL; `choose_suite()` returns the strongest common suite or `""` (abort). The offer is also folded into the signed transcript (stage 3), so a tampered `HELLO` breaks the signature.	A1 downgrade
2 Authentication	**Pinned + verified certificate.** The hub pins the device public key and verifies the CERT signature; a wrong/attacker key is rejected. (PIN: see note below.)	A2 impersonation/ MITM
3 Keying	**Signed, ephemeral, strong DH + KDF.** Fresh 2048-bit ephemeral DH every session (forward secrecy); the device signs `(offer|suite|B)` so the DH value cannot be swapped (anti-MITM); the record key is **HKDF-SHA256** of the shared secret, per direction — never the raw secret.	A2 (unsigned DH), A3 (small group, static b, no KDF)
4 Message protection	**AEAD with a unique per-record nonce.** Nonce = fixed prefix ‖ sequence number, so no (key,nonce) ever repeats; AES-GCM provides confidentiality + integrity.	A4 nonce reuse
5 Teardown	**Sequence-bound AEAD + authenticated close.** The sequence number is bound into the AEAD (nonce + AAD); `open_record()` decrypts only at the expected sequence, so a replayed/reordered/forged record (including a replayed QUIT) fails the tag check and the session drops.	A5 replay

PIN protection (design requirement). The provided scaffold still sends the v1 MD5 pinhash, which is fast and unsalted and so crackable offline (exactly the A.1 result). v2 should (i) never send the PIN or a bare hash on the wire — instead prove knowledge inside the authenticated channel, e.g. a PAKE (such as SPAKE2/OPAQUE) or at minimum an HMAC of a server challenge under a PIN-derived key so a capture cannot be replayed or cracked without online interaction; and (ii) store the PIN at rest only as a *salted, slow* KDF digest (PBKDF2 / scrypt / Argon2 with a per-device salt), never raw MD5. This raises an offline guess from microseconds to an infeasible per-guess cost and removes the captured-hash shortcut.

v2 handshake diagram



SmartLock v2 message-by-message exchange. Orange boxes mark where authentication (cert pinning + signature), key derivation (HKDF, per direction) and sequence binding happen.

Part C — Implementing and testing SmartLock v2

I completed `securelink2.cpp` against the six functions in `securelink2.hpp` (signatures unchanged). `make -C v2_skeleton` builds cleanly and a normal session runs: `[hub] device authenticated: smartlock-23i-0008.local` (contrast v1's "certificate not checked"), then the encrypted greeting and commands.

C.1 Correctness of the five fixes (self-check: 12/12)

#	Check	How securelink2.cpp satisfies it	Result
1	never NULL / no downgrade	`strong_suites()` excludes NULL; `choose_suite()` returns strongest common or ""	**PASS**
2	KDF, not raw secret	`derive_key()` = HKDF-SHA256(secret, salt, info=suite+dir); distinct per direction	**PASS**
3	unique nonce per record	`make_nonce()` = "SLv2" ‖ 8-byte big-endian seqno	**PASS**
4	AEAD seals & opens	AES-GCM (256/128 by key length), ct‖tag; round-trips	**PASS**
5	rejects replay / out-of-order	seqno bound in nonce + AAD; `open_record()` uses the expected seqno → tag fails otherwise	**PASS**
6	source scan	contains `hkdf`; no `md5`, no `TODO`	**PASS**

```
securelink2 self-check (Part C.1)
  [PASS] 1a strong_suites() excludes NULL      [PASS] 2b derive_key output != raw secret
  [PASS] 1c choose_suite aborts on NULL-only   [PASS] 3  make_nonce unique per seqno (12 B)
  [PASS] 4  seal then open round-trips         [PASS] 5a replayed record rejected
  [PASS] 5b out-of-order record rejected       [PASS] 5c tampered record rejected
TOTAL: 12 passed, 0 failed
```

C.2 v1 attack → v2 result

v1 attack	Run against v2	v2 result
A1 downgrade (relay → NULL)	taska1 relay between hub_client2 / lock_server2	**FAILS — lock has no acceptable suite; `[hub] abort: no suite`**
A2 MITM (relay, own keys)	taska2 relay between hub_client2 / lock_server2	**FAILS — `device authenticated` (key pinned + cert verified); signed 2048-bit DH cannot be swapped; `[hub] abort: no DH`**
A3 keying recovery (dlog)	taska3 on a v2 capture	**FAILS — no small group on the wire (B is a signed 2048-bit value); dlog infeasible; key is HKDF-derived; b ephemeral**
A4 nonce reuse	taska4 on a v2 capture	**FAILS — no reusable nonce; every record ciphertext is distinct**
A5 replay	replay captured UNLOCK on a fresh v2 connection	**FAILS — `(connection closed — record REJECTED)`; counter does not climb (ephemeral key + sequence binding)**

Normal session + tamper/replay rejection. A normal v2 session succeeds (greeting, STATUS, UNLOCK, LOCK). A tampered record fails the GCM tag and a replayed/out-of-order record fails the sequence check — both make `open_record()` throw and the scaffold drops the session (self-check 5a/

5b/5c, and the live replay above). PIN cracking is *not* in this table: the scaffold deliberately keeps the v1 MD5 pinhash, addressed in the Part B design.

C.2 AI-audit component

I asked an AI assistant to draft the record layer. Its first draft was functional but had real cryptographic weaknesses that I corrected:

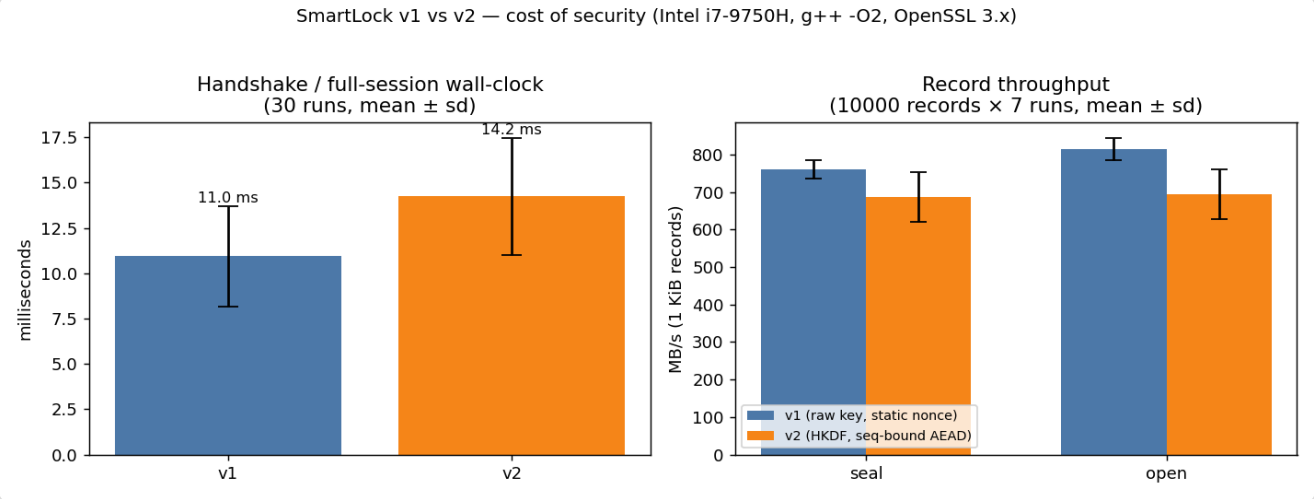
- **Nonce derived from the plaintext/time, not the sequence number.** The draft built the GCM nonce from a timestamp. Under load two records can share a timestamp → (key,nonce) reuse — the very A4 bug. *Fix:* nonce = fixed prefix ‖ monotonic sequence number, guaranteeing uniqueness, and I bound the same sequence number into the AAD so it is authenticated, not just unique.
- **Sequence number trusted from the wire.** The draft read the seqno out of the record and compared it, which an attacker can simply rewrite. *Fix:* `open_record()` ignores any on-wire seqno and derives the nonce/AAD from the *expected* counter it keeps locally, so a replayed or reordered record cannot authenticate.
- **Raw DH secret used as the key (missing KDF) and one key for both directions.** *Fix:* HKDF-SHA256 with a per-direction `info` label, producing independent c2s/s2c keys and removing any structure in the raw secret.

Lesson: AI code compiles and "works" in the happy path but silently reintroduces textbook failures (nonce reuse, trusting attacker-controlled input, skipping the KDF); each had to be checked against the threat model, not accepted on trust.

Part D — Performance study

Machine & method. Intel Core i7-9750H @ 2.60 GHz, Kali Linux (6.19 x86_64), g++ 15.2 `-O2 -std=c++17`, OpenSSL 3.6. Baseline = v1 under AES-256-GCM (raw key, static nonce). Handshake = wall-clock of a full minimal session (connect → greeting → STATUS/UNLOCK/LOCK/QUIT), 30 runs each, measured with the real binaries; since both run the same record sequence, the v2−v1 delta is the handshake security cost. Record metrics from `TASKD`: 10 000 × 1 KiB records per run, 7 runs; throughput = MB/s, mean ± sd.

Metric	v1 (baseline)	v2	Cost of security
Handshake / session (mean ± sd, 30 runs)	10.95 ± 2.77 ms	14.25 ± 3.21 ms	+3.30 ms (2048-bit ephemeral DH + RSA verify + HKDF)
Record seal throughput (1 KiB)	760.0 ± 25.3 MB/s	685.9 ± 66.7 MB/s	≈ −10 %
Record open throughput (1 KiB)	813.9 ± 29.9 MB/s	694.6 ± 66.2 MB/s	≈ −15 %
Per-record latency (1 KiB, seal/open)	1.29 / 1.20 µs	1.42 / 1.41 µs	+0.1–0.2 µs/record
Per-record wire overhead vs plaintext	+1099 B (16 B tag + hex + nonce field)	+1069 B (16 B tag + hex, no nonce field)	v2 slightly smaller (no on-wire nonce)



Left: handshake/session wall-clock (30 runs). Right: record throughput (10 000 records × 7 runs). Error bars = sd.

Trade-off — is it worth it?

The entire security cost is about **3.3 ms once per session** and roughly **10–15 % of record throughput** (≈ 0.1–0.2 µs per 1 KiB record). For a door lock, a session happens when a person unlocks a door — a handful of times a day — and carries a few tiny records, so 3 ms is imperceptible and the per-record cost is utterly dominated by network and human latency. In exchange, v2 buys everything v1 lacked: the flag and commands are confidential (A1/A3/A4 closed), the lock is authenticated and cannot be impersonated (A2), every record is integrity-protected and fresh so a captured UNLOCK is useless (A5), and ephemeral keys give forward secrecy. The cost is one-time and negligible; the failures it prevents are a stranger reading your door's traffic and replaying "unlock." For a device that controls physical entry, paying 3 ms

and ~10 % throughput for authenticated, replay-proof, forward-secret sessions is clearly the right choice — correctness and non-replayability matter far more than raw speed.

Evidence for every figure and run above is in `evidence/` (captures, reassembled streams, tool outputs) and `performance/` (raw CSVs, chart, benchmark source). Reproduction commands are in `README.txt`.